

File-Based Persistent Conversation History

1. Main Goal

Until now, we used:

InMemoryChatMessageHistory

This stores conversation history in application memory.

But if the Python program stops:

Program Stops



In-Memory Data Is Lost

Now we want:

Program Stops



Conversation History Still Exists



Program Starts Again



Previous History Can Be Loaded

This is:

Persistent Conversation History

2. Main Idea

We will store conversation history inside JSON files.

Concept:

Session



Conversation Messages



Convert Messages to Dictionaries



Save as JSON



File

Later:

JSON File



Read Data



Convert Dictionaries Back to Messages



Conversation Continues

3. Complete Architecture

User



session_id



FileChatMessageHistory



JSON File



Load Previous Messages



RunnableWithMessageHistory



LCEL Chain



AI Response



Save Updated Messages



JSON File

4. Concepts Required Before Main Program

Before understanding the complete program, students should understand:

`os.makedirs()`

Classes

`@property`

File Read / Write

JSON

Serialization

BaseChatMessageHistory

5. Creating a Folder with os.makedirs()

Example:

```
import os
```

```
folder_name = "chat_history2"
```

```
os.makedirs(  
    folder_name,  
    exist_ok=True  
)
```

```
print("Folder created")
```

6. What Does os.makedirs() Do?

```
os.makedirs(...)
```

creates a directory or folder.

Example:

```
os.makedirs(  
    "chat_history2",  
    exist_ok=True
```

)

Concept:

Python Program



`os.makedirs()`



Create Folder



`chat_history2`

7. What Is `exist_ok=True`?

Suppose the folder already exists.

Without proper handling, creating the same folder again may cause an error.

Using:

`exist_ok=True`

means:

Folder Does Not Exist



Create It

Folder Already Exists



Do Not Raise Error

8. Simple Rule

`os.makedirs()`



Create Folder

`exist_ok=True`



Existing Folder Is Okay

9. Normal Class Method

Consider:

`class Student:`

`def get_name(self):`

`return "Durga"`

```
s = Student()
```

```
print(  
    s.get_name()  
)
```

Output:

Durga

Here:

```
get_name()
```

is a normal method.

So we call it using:

```
s.get_name()
```

Notice:

```
()
```

are required.

10. Method Call

```
s.get_name()
```

means:

Call the method



Execute method body



Return "Durga"

11. Using @property

Now consider:

```
class Student:
```

```
    @property
```

```
    def get_name(self):
```

```
        return "Ravi"
```

```
s = Student()
```

```
print(
```

```
    s.get_name
```

```
)
```

Output:

12. What Is @property?

@property is a Python decorator.

It allows us to access a method like a normal attribute.

Normally:

```
s.get_name()
```

With @property:

```
s.get_name
```

13. Easy Comparison

Normal method:

```
class Student:
```

```
    def get_name(self):  
        return "Durga"
```

Call:

```
s.get_name()
```

With property:

```
class Student:
```

```
@property
def get_name(self):
    return "Ravi"
```

Access:

s.get_name

14. Easy Memory Rule

Normal Method

↓

Use ()

@property

↓

Access Like Attribute

15. Why Do We Need @property in Our Chat History Class?

Later we define:

```
@property
def messages(self):
```

Because the chat-history abstraction expects us to access history like:

`history.messages`

not:

`history.messages()`

So:

`@property`



Method behaves like attribute

16. File Writing

Example:

with open(

 "student1.txt",

 "w",

 encoding="utf-8"

) as file:

 file.write(

 "Durga is learning Python with full focus"

)

Here:

"w"



Write Mode

17. File Reading

```
with open(  
    "student1.txt",  
    "r",  
    encoding="utf-8"  
) as file:
```

```
    data = file.read()
```

```
print(data)
```

Output:

Durga is learning Python with full focus

18. File Modes

"w"



Write

"r"



Read

19. What Does with open(...) Do?

We use:

with open(...) as file:

to safely work with files.

Concept:

Open File



Use File



Automatically Close File

20. Why encoding="utf-8"?

```
encoding="utf-8"
```

helps handle Unicode text properly.

This is useful when storing normal text, symbols, and many languages.

21. Main Program — File-Based Persistent Chat History

```
import os
```

```
import json
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import (
```

```
    ChatPromptTemplate,
```

```
    MessagesPlaceholder
```

```
)
```

```
from langchain_core.chat_history import (
```

```
    BaseChatMessageHistory
```

```
)
```

```
from langchain_core.messages import (
    BaseMessage,
    message_to_dict,
    messages_from_dict
)
```

```
from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)
```

```
# =====
# 1. PERSISTENT CHAT HISTORY
# =====
```

```
class FileChatMessageHistory(
    BaseChatMessageHistory
):

    def __init__(
```

```
self,  
session_id,  
storage_path="chat_history"  
):
```

```
self.session_id = session_id
```

```
os.makedirs(  
    storage_path,  
    exist_ok=True  
)
```

```
self.file_path = os.path.join(  
    storage_path,  
    f"{session_id}.json"  
)
```

```
# -----
```

```
# LOAD MESSAGES
```

```
# -----
```

```
@property
```

```
def messages(self):
```

```
    if not os.path.exists(
```

```
        self.file_path
```

```
    ):
```

```
        return []
```

```
    with open(
```

```
        self.file_path,
```

```
        "r",
```

```
        encoding="utf-8"
```

```
    ) as file:
```

```
        data = json.load(
```

```
            file
```

```
        )
```

```

    return messages_from_dict(
        data
    )

# -----
# SAVE MESSAGES
# -----

def add_messages(
    self,
    messages: list[BaseMessage]
) -> None:

    all_messages = list(
        self.messages
    )

    all_messages.extend(
        messages
    )

```

```
)
```

```
serialized = [  
    message_to_dict(message)  
    for message in all_messages  
]
```

```
with open(  
    self.file_path,  
    "w",  
    encoding="utf-8"  
) as file:
```

```
    json.dump(  
        serialized,  
        file,  
        ensure_ascii=False,  
        indent=2  
    )
```

```
# -----
```

```
# CLEAR HISTORY
```

```
# -----
```

```
def clear(self) -> None:
```

```
    with open(
        self.file_path,
        "w",
        encoding="utf-8"
    ) as file:
```

```
        json.dump(
            [],
            file
        )
```

```
# =====
```

2. CREATE MODEL

=====

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

=====

3. CREATE PROMPT

=====

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            """"  
  
            You are a helpful AI assistant.  
  
            Answer in simple English.            """)
```

```

        """
    ),

    MessagesPlaceholder(
        variable_name="history"
    ),

    (
        "human",
        "{question}"
    )
]
)

# =====
# 4. CREATE LCEL CHAIN
# =====

chain = (

```

```

    prompt
    | llm
)

# =====

# 5. GET SESSION HISTORY

# =====

def get_session_history(
    session_id
):

    return FileChatMessageHistory(
        session_id
    )

# =====

# 6. ADD MESSAGE HISTORY

```

```
# =====
```

```
chatbot = RunnableWithMessageHistory(
```

```
    chain,
```

```
    get_session_history,
```

```
    input_messages_key="question",
```

```
    history_messages_key="history"
```

```
)
```

```
# =====
```

```
# 7. GET SESSION ID
```

```
# =====
```

```
session_id = input(
```

```
    "Enter session id: "
```

```
)
```

```
print("\nAI Chatbot")  
print("Type 'exit' to stop")  
print("-----")
```

```
# =====  
# 8. CHAT LOOP  
# =====
```

```
while True:
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    if user_input.lower() == "exit":
```

```
print(  
    "Chat ended."  
)  
  
break  
  
response = chatbot.invoke(  
  
    {  
        "question": user_input  
    },  
  
    config={  
        "configurable": {  
            "session_id": session_id  
        }  
    }  
)
```

```
print(  
    "AI:",  
    response.content  
)
```

22. What Are We Creating?

We create our own class:

```
class FileChatMessageHistory(  
    BaseChatMessageHistory  
):
```

This means:

FileChatMessageHistory



inherits from



BaseChatMessageHistory

23. Why BaseChatMessageHistory?

BaseChatMessageHistory represents the basic contract for a chat-history implementation.

Our class should provide behaviour for:

Read Messages

Add Messages

Clear Messages

In our implementation:

messages

add_messages()

clear()

24. Constructor

```
def __init__(  
    self,  
    session_id,  
    storage_path="chat_history"  
):
```

The constructor receives:

session_id

and optionally:

storage_path

Default:

chat_history

25. Store the Session ID

```
self.session_id = session_id
```

Suppose:

```
session_id = "durga"
```

then:

```
self.session_id
```

↓

```
"durga"
```

26. Create Storage Folder

```
os.makedirs(  
    storage_path,  
    exist_ok=True  
)
```

If the folder does not exist:

Create chat_history Folder

If it already exists:

Continue Without Error

27. Create One File for Every Session

```
self.file_path = os.path.join(  
    storage_path,  
    f"{session_id}.json"  
)
```

Suppose:

```
session_id = "durga"
```

Then:

```
chat_history/durga.json
```

Suppose:

```
session_id = "ravi"
```

Then:

```
chat_history/ravi.json
```

28. Session Architecture

chat_history/

|

└─ durga.json

|

└─ ravi.json

|

└─ suresh.json

|

└─ student1.json

So:

One Session

↓

One JSON File

29. Loading Messages

We define:

@property

def messages(self):

This means we access it like:

history.messages

not:

history.messages()

30. First Check — Does File Exist?

```
if not os.path.exists(
```

```
    self.file_path
```

```
):
```

```
    return []
```

Suppose the user starts a new session.

Example:

durga.json

does not exist

Then:

```
return []
```

means:

No Previous Conversation

31. Loading Existing JSON Data

If the file exists:

```
with open(
```

```
self.file_path,  
"r",  
encoding="utf-8"  
) as file:
```

```
data = json.load(  
    file  
)
```

Here:

```
json.load(file)
```

reads JSON from the file and converts it into Python data.

32. json.load() vs json.loads()

Previously we learned:

```
json.loads(json_string)
```

which means:

JSON String



Python Object

Here we use:

`json.load(file)`

which means:

JSON File



Python Object

Easy difference:

`loads()`



String

`load()`



File

33. Convert Dictionaries Back into Messages

After reading:

`data = json.load(file)`

we get Python dictionaries.

Then:

```
return messages_from_dict(  
    data
```

)

converts them back into:

HumanMessage

AIMessage

HumanMessage

AIMessage

34. Loading Flow

JSON File



json.load()



Python Dictionaries



messages_from_dict()



LangChain Message Objects

35. Adding Messages

Our class defines:

```
def add_messages(  
    self,  
    messages: list[BaseMessage]  
) -> None:
```

This method receives new messages that need to be stored.

36. What Is BaseMessage?

BaseMessage is the common base type for LangChain message objects.

Examples include:

HumanMessage

AIMessage

SystemMessage

So:

```
list[BaseMessage]
```

means:

A list containing LangChain messages

37. First Load Existing Messages

```
all_messages = list(  

```

```
self.messages  
)
```

This loads previous messages from the JSON file.

Concept:

Existing File



Previous Messages



all_messages

38. Add New Messages

```
all_messages.extend(  
    messages  
)
```

Suppose existing history contains:

HumanMessage 1

AIMessage 1

and new messages contain:

HumanMessage 2

AIMessage 2

After:

```
all_messages.extend(messages)
```

we get:

HumanMessage 1

AIMessage 1

HumanMessage 2

AIMessage 2

39. append() vs extend()

Very important.

Suppose:

```
a = [10, 20]
```

```
b = [30, 40]
```

Using:

```
a.append(b)
```

produces:

```
[10, 20, [30, 40]]
```

But:

```
a.extend(b)
```

produces:

```
[10, 20, 30, 40]
```

Here we need:

```
extend()
```

because we want to add all new message objects into the existing list.

40. Serialize All Messages

```
serialized = [  
    message_to_dict(message)  
    for message in all_messages  
]
```

This converts:

LangChain Messages



Python Dictionaries

41. Save to JSON File

```
with open(  
    self.file_path,  
    "w",
```

```
        encoding="utf-8"  
) as file:  
  
    json.dump(  
        serialized,  
        file,  
        ensure_ascii=False,  
        indent=2  
    )
```

Here:

```
json.dump()  
writes Python data directly into a JSON file.
```

42. `json.dump()` vs `json.dumps()`

Previously:

```
json.dumps(data)
```

means:

Python Object



JSON String

Now:

```
json.dump(  
    data,  
    file  
)
```

means:

Python Object



JSON File

Easy memory:

dumps



String

dump



File

43. Four Important JSON Functions

json.dumps()

Python → JSON String

`json.loads()`

JSON String → Python

`json.dump()`

Python → JSON File

`json.load()`

JSON File → Python

44. Student Memory Trick

"s" at the end



String

So:

dumps

loads



String Operations

Without s:

dump

load



45. What Is `ensure_ascii=False`?

`ensure_ascii=False`

allows Unicode characters to be stored in a more readable form.

This is useful when conversation text contains:

Telugu

Hindi

Special Characters

Other Unicode Text

46. What Is `indent=2`?

`indent=2`

formats the JSON nicely.

Instead of difficult one-line JSON, we get readable formatted JSON.

47. Clear Conversation History

Our class provides:

`def clear(self) -> None:`

Inside:

```
json.dump(
```

```
    [],
```

```
    file
```

```
)
```

So the JSON file becomes an empty list.

Concept:

Existing History

↓

`clear()`

↓

`[]`

48. Important Point About `clear()`

We are not necessarily deleting the file.

We are replacing its conversation content with:

`[]`

So:

File Still Exists



History Becomes Empty

49. Create Model

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

This is the model used by our chatbot.

50. Create Prompt

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            ""  
              
            You are a helpful AI assistant.  
            Answer in simple English.        )  
    ]
```

```

        """"
    ),

    MessagesPlaceholder(
        variable_name="history"
    ),

    (
        "human",
        "{question}"
    )
]
)

```

Prompt structure:

System Message



Previous Conversation History



Current Question

51. Create LCEL Chain

```
chain = (  
    prompt  
    | llm  
)
```

Concept:

Prompt



Model

This is a Runnable.

52. get_session_history()

```
def get_session_history(  
    session_id  
):  
  
    return FileChatMessageHistory(  
        session_id  
    )
```

Suppose:

```
session_id = "durga"
then:
FileChatMessageHistory(
    "durga"
)
uses:
chat_history/durga.json
```

53. Different Sessions

session_id = "durga"



durga.json

session_id = "ravi"



ravi.json

Therefore:

Different Session IDs



Different JSON Files



Different Conversation Histories

54. Add RunnableWithMessageHistory

```
chatbot = RunnableWithMessageHistory(
```

```
    chain,
```

```
    get_session_history,
```

```
    input_messages_key="question",
```

```
    history_messages_key="history"
```

```
)
```

This wraps our LCEL chain with message-history management.

55. Role of input_messages_key

```
input_messages_key="question"
```

means:

Current User Input



Stored under key



question

Invocation:

```
{  
  "question": user_input  
}
```

56. Role of history_messages_key

history_messages_key="history"

matches:

```
MessagesPlaceholder(  
  variable_name="history"  
)
```

So:

Stored Previous Messages



history



MessagesPlaceholder

57. Get Session ID

```
session_id = input(  
    "Enter session id: "  
)
```

Example:

Enter session id: durga

Now this conversation uses:

durga.json

58. Chat Loop

while True:

allows continuous conversation.

The user enters:

Question 1

Question 2

Question 3

...

until:

exit

59. Chat Invocation

```
response = chatbot.invoke(  
  
    {  
        "question": user_input  
    },  
  
    config={  
        "configurable": {  
            "session_id":  
                session_id  
        }  
    }  
)
```

The important parts are:

question



Current Input

and:

session_id



Which History File?

60. Complete Runtime Flow

Suppose:

session_id = "durga"

and user asks:

What is my name?

Flow:

Current User Question



session_id = durga



FileChatMessageHistory("durga")



durga.json



Load Previous Messages



Insert into Prompt



Call Model



Get AI Response



Save New Messages



durga.json Updated

61. Persistence Demonstration

This is the most important classroom demonstration.

First Run

Enter:

Enter session id: durga

Then:

You: My name is Durga.

Close the Python program.

Second Run

Start the program again.

Use the same:

Enter session id: durga

Ask:

You: What is my name?

Because the conversation was saved in:

durga.json

the previous history can be loaded again.

That is persistent conversation history.

62. Same Session After Restart

First Program Run



session_id = durga



durga.json



Save Conversation



Program Stops

Later:

Second Program Run



session_id = durga



Read durga.json



Previous History Restored

63. Different Session

If later we use:

session_id = ravi

then:

ravi.json

is used.

So:

durga.json

≠

ravi.json

Each session has separate persistent conversation history.

64. In-Memory vs File-Based History

In-Memory

Stored temporarily in application memory

May disappear when program stops

Very easy

Good for simple demos

InMemoryChatMessageHistory

File-Based

Stored in JSON file

Can survive program restart

Requires file handling

Demonstrates persistence

Custom
FileChatMessageHistory

65. Serialization Flow Inside Our Class

When saving:

LangChain Messages



message_to_dict()



Python Dictionaries



`json.dump()`



JSON File

66. Deserialization Flow Inside Our Class

When loading:

JSON File



`json.load()`



Python Dictionaries



`messages_from_dict()`



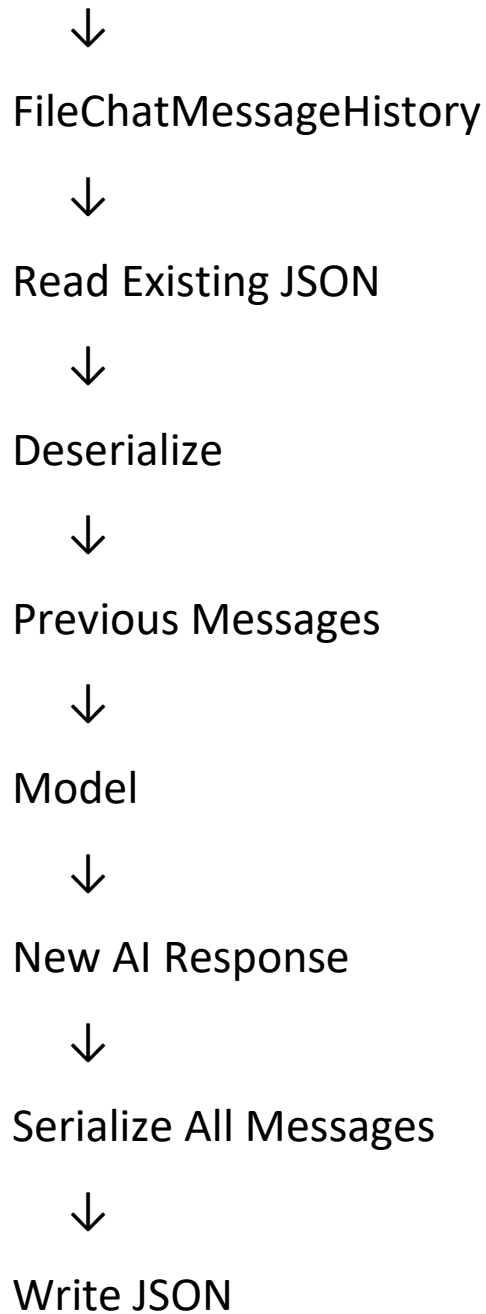
LangChain Messages

67. Complete Persistence Cycle

User Message



`RunnableWithMessageHistory`



68. Important Methods in FileChatMessageHistory

Method / Property Purpose

<code>__init__()</code>	Prepare session and file path
-------------------------	-------------------------------

<code>messages</code>	Load previous messages
<code>add_messages()</code>	Add and save new messages
<code>clear()</code>	Clear stored conversation

69. Important Python Functions Used

Function	Purpose
<code>os.makedirs()</code>	Create history folder
<code>os.path.join()</code>	Build file path
<code>os.path.exists()</code>	Check if file exists
<code>open()</code>	Read/write file
<code>json.load()</code>	JSON file → Python
<code>json.dump()</code>	Python → JSON file
<code>message_to_dict()</code>	LangChain Message → Dictionary
<code>messages_from_dict()</code>	Dictionaries → LangChain Messages

70. Common Mistake — Thinking @property Is a Normal Method Call

Wrong:

`history.messages()`

In our implementation:

```
@property
```

```
def messages(self):
```

Correct:

```
history.messages
```

71. Common Mistake — Confusing `json.load()` and `json.loads()`

Remember:

```
json.load()
```



Read From File

```
json.loads()
```



Read From String

72. Common Mistake — Confusing `json.dump()` and `json.dumps()`

Remember:

```
json.dump()
```



Write To File

```
json.dumps()
```



Create JSON String

73. Common Mistake — Forgetting to Load Existing History

If we directly save only new messages without loading previous messages, old conversation may be lost.

That is why we first do:

```
all_messages = list(  
    self.messages  
)
```

Then:

```
all_messages.extend(  
    messages  
)
```

74. Common Mistake — Using `append()` Instead of `extend()`

Suppose messages itself is a list.

Using:

```
all_messages.append(  
    messages  
)
```

would add the whole list as one item.

Instead:

```
all_messages.extend(  
    messages  
)
```

adds each new message to the history list.

75. Common Mistake — Same File for Every Session

If all users use the same file:

history.json

their histories may get mixed.

Our design uses:

durga.json

ravi.json

student1.json

So:

Session ID



Separate File

76. Important Key Points

1. Persistent conversation history survives beyond the current program execution.
2. We can use JSON files as a simple persistent storage mechanism.
3. FileChatMessageHistory inherits from BaseChatMessageHistory.
4. Every session can have a separate JSON file.
5. @property allows messages to be accessed like an attribute.
6. messages loads history from the JSON file.
7. add_messages() loads existing history, adds new messages, and writes everything back.
8. message_to_dict() serializes LangChain messages.
9. messages_from_dict() restores LangChain messages.
10. json.dump() writes Python data to a JSON file.
11. json.load() reads Python data from a JSON file.

12. `RunnableWithMessageHistory` uses `get_session_history()` to obtain the correct history implementation.
 13. Same session ID points to the same history file.
 14. Different session IDs create separate histories.
 15. File-based history allows a conversation to continue even after restarting the Python program.
-

77. Five Coding Lines Students Must Remember

1. Create Folder

```
os.makedirs(  
    storage_path,  
    exist_ok=True  
)
```

2. Create Session File Path

```
self.file_path = os.path.join(  
    storage_path,  
    f"{session_id}.json"  
)
```

3. Read JSON

```
data = json.load(  
    file  
)
```

4. Convert Back to Messages

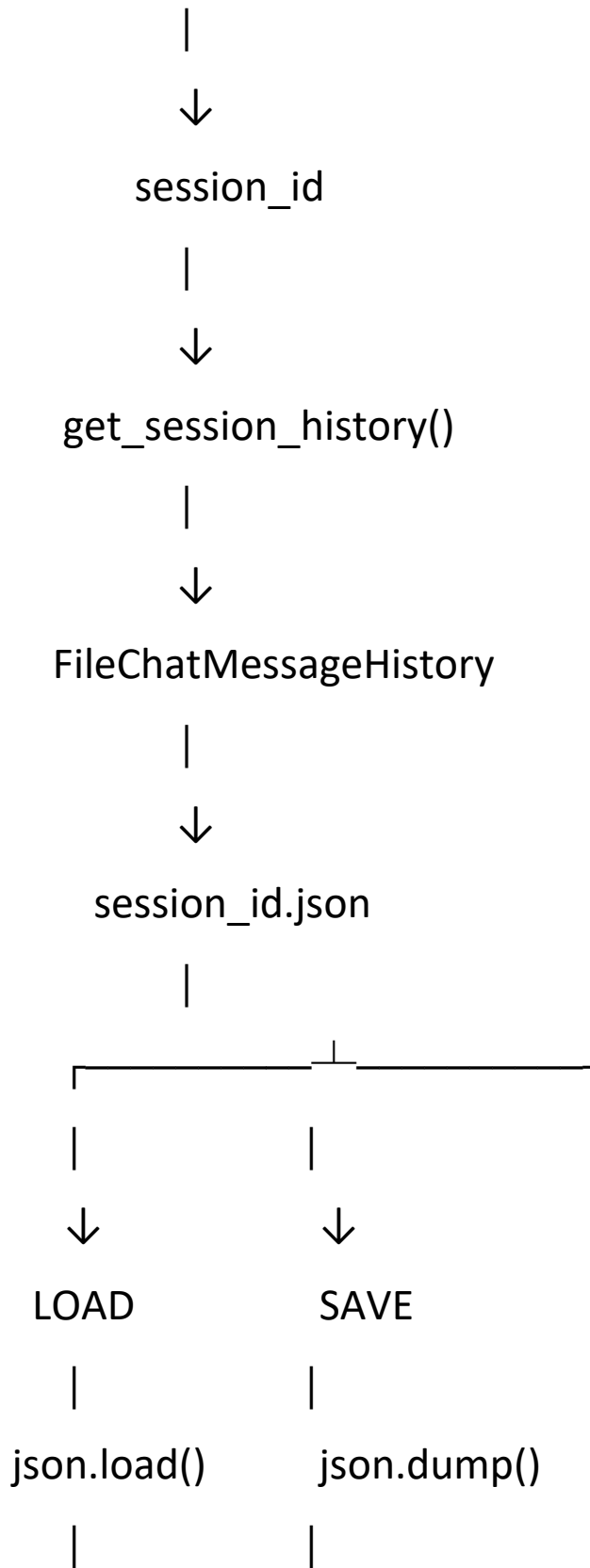
```
messages_from_dict(  
    data  
)
```

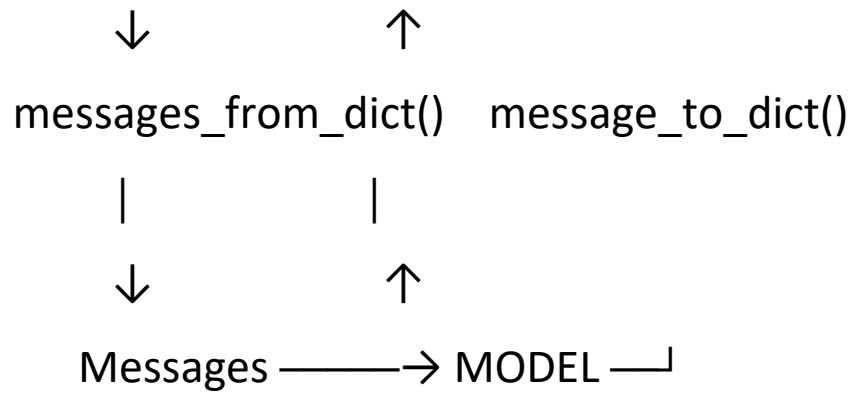
5. Save Serialized Messages

```
json.dump(  
    serialized,  
    file,  
    ensure_ascii=False,  
    indent=2  
)
```

78. Final Master Diagram

USER





79. Easy Memory Formula

SAVE

====

Messages



`message_to_dict()`



Dictionary



`json.dump()`



JSON File

LOAD

====

JSON File



`json.load()`



Dictionary



`messages_from_dict()`



Messages

80. Final Memory Rule

`session_id`



JSON File



Persistent History

Example:

durga

↓
durga.json
↓
Durga Conversation
ravi
↓
ravi.json
↓
Ravi Conversation

81. One-Line Conclusion

File-based persistent conversation history stores each session's LangChain messages in a JSON file, allowing the application to load the same conversation again even after the Python program has stopped and restarted.